Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 5° Appello – 10/09/2018 – a.a. 2017/2018	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA** | | | | | | |	

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 12 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 4] Data la seguente definizione di struttura di tipo albero:

```
struct tree { int dato; tree *sx, *dx; } tree;
```

Scrivere 2 funzioni `cerca(tree* root)` che restituiscano il valore massimo contenuto nei nodi di un albero binario con radice `root`:

A. Se l'albero NON è stato creato in modalità BST
B. Se l'albero è un BST

Dire nel caso A e nel caso B quale è il tempo per questa ricerca T(N), motivando la risposta.

[Domanda 2 - punti 2] Sia A una matrice 5x4 allocata dinamicamente. Far vedere come è possibile accedere all'elemento A[3][2] utilizzando l'aritmetica dei puntatori, spiegando in dettaglio come operano i puntatori nel caso di accesso ad un elemento di una matrice.

[Domanda 3 - punti 4] Dato un array di caratteri (stringa) definire e scrivere una funzione `filtra` **che stampi un array di caratteri (stringa) che contiene solo le lettere minuscole presenti nella stringa passata:**

```
char frase[50] = "Questo è un esercizio MOLTO facile!!";
/* output atteso: uestouneserciziofacile */
```

[Domanda 4 - punti 2] cosa vale 1101_2 se il numero è:

numero intero [ ]
numero intero con segno [ ]
numero intero in complemento a 2 [ ]
Mostrare in dettaglio tutti i passaggi

PARTE PRATICA: 21 PUNTI – PUNTEGGIO MINIMO 11 PUNTI

[A - punti 3] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
#define DIM 2
typedef enum Trichiesta { WEB, TELEFONO, UFFICIO } Trichiesta;
typedef struct Tordine{
  char materiale[20];
  int quantita;
  float prezzo;
  Trichiesta tipoRichiesta;
  Tordine () { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto E */ }
} Tordine;
typedef struct Tnodo {
   Tordine dato;
   Tnodo* next;
   Tnodo* prev;
   Tnodo () { /*  da implementare */ }
   Tnodo (Tordine o, Tnodo* n, Tnodo* p) { /*  da implementare */ }
   void stampa() { /*  da implementare */ }
} Tnodo;
```

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
Tnodo* negozio[DIM];
for (int i=0; i<DIM; i++) { negozio[i] = NULL; }
Tordine o;
for (int i=0; i<5; i++) { newOrdine(&o); addOrdine(negozio, DIM, o); }
stampaNegozi(negozio, DIM);
o = cercaOrdine(negozio, DIM);
o.stampa();
```

[C - punti 4] Implementare una funzione **`newOrdine`** che riceve in input per riferimento una variabile di tipo **`Tordine`** **o** e la inizializza con:

- campo **`tipoRichiesta`** valore casuale tra WEB, TELEFONO, UFFICIO;
- campo **`quantita`** valore letto da tastiera, tra 1 e 20, con controllo input.
- campo **`prezzo`** valore casuale tra 10.00 e 99.99;
- campo **`materiale`** letta da tastiera.

[D - punti 4] Implementare una funzione **`addOrdine`** che riceve come input una variabile di tipo array **`Tnodo*`** **neg**, la dimensione dell'array **dim** e variabile di tipo **`Tordine`** **o** e la inserisce nella listaFIFO **neg[x]** (con **x** valore casuale tra 0 e 1) (**`insertFirst`**).

[E - punti 4] Implementare una funzione **`stampaNegozi`** che riceve in input una variabile di tipo array **`Tnodo`** *** neg**, la dimensione dell'array **dim** e stampa il contenuto delle liste FIFO utilizzando opportunamente il metodo **stampa di** **`Tnodo`**. Nella forma (per enum stampare etichetta):

`materiale` **QTA=**`quantita` **(**`tipoRichiesta`**)** **prezzo=**`prezzo`

[F - punti 5] Implementare una funzione **`cercaOrdine`** che riceve come input una variabile di tipo array **`Tnodo*`** **neg** e la dimensione dell'array **dim** e cerca l'ordine che ha il maggior costo medio (`prezzo/quantita`). La funzione, oltre a restituire l'ordine trovato, aggiunge l'ordine stesso ad un file denominato **`"costoso.txt"`** nella forma:

`materiale` **QTA=**`quantita` **(**`tipoEstrazione`**)** **prezzo=**`prezzo`

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!